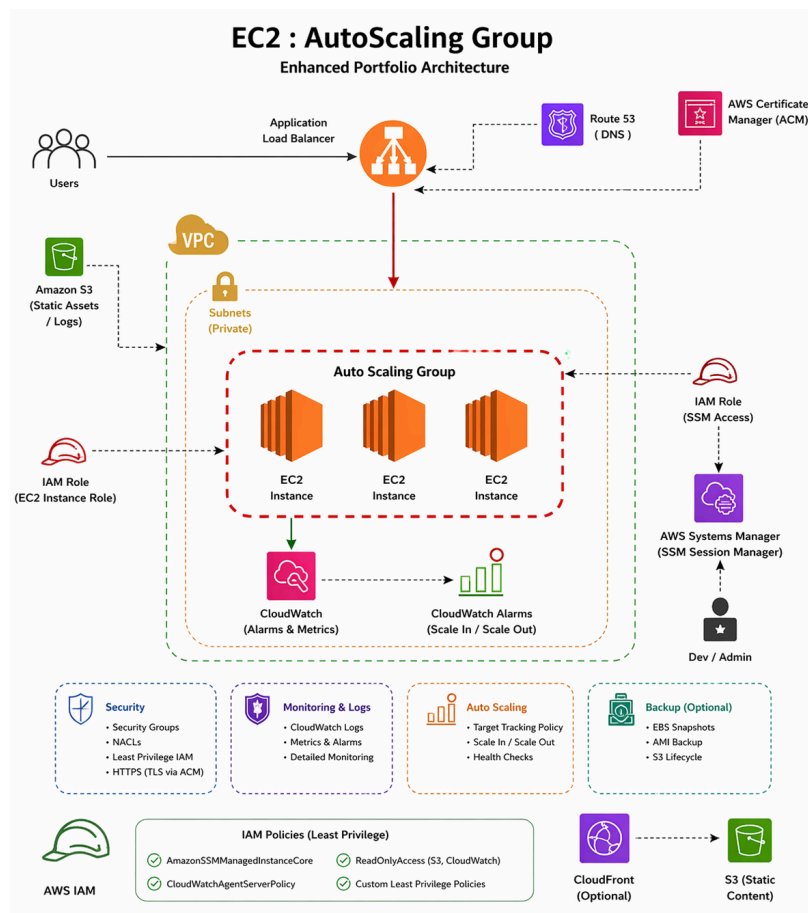


<https://www.youtube.com/watch?v=fwfkSxb1T-s&list=PL7iMyoQPMtAN4xl6oWzafqJebfay7K8KP&index=10>



Assuming a lean portfolio build like this diagram: 1 small EC2 app server in an Auto Scaling Group, 1 Application Load Balancer, 1 Route 53 hosted zone, 1 ACM certificate, a small gp3 EBS volume, a few CloudWatch alarms, a tiny S3 bucket, and SSM access. In that setup, ASG, IAM, VPC, security groups, SSM, and ACM public certs have no extra service charge, while EC2, ALB, Route 53 hosted zones/queries, CloudWatch alarms, S3 storage, and EBS storage/snapshots are the main metered costs.

1) Users → Application Load Balancer (ALB)

What it does: gives the app a single public entry point and distributes traffic across healthy instances.

Why it is there: makes the site look professional and lets the ASG scale behind one DNS name.

What it prevents: a single EC2 instance becoming a bottleneck or single point of failure.

Cost: \$0.0225/hr per ALB plus \$0.008 per LCU-hour; for a small portfolio demo, this is usually the biggest ongoing cost.

2) Route 53

What it does: maps your domain name to the ALB.

Why it is there: so you use myproject.com instead of a raw AWS URL or IP.

What it prevents: hardcoded IPs and ugly manual endpoint changes when the backend changes.

Cost: \$0.50 per hosted zone per month and \$0.40 per million standard queries; alias queries to AWS load balancers are included at no extra query cost.

3) AWS Certificate Manager (ACM)

What it does: provides the TLS certificate for HTTPS.

Why it is there: makes the portfolio project look production-ready.

What it prevents: plain-text traffic and the hassle of manually renewing certs.

Cost: public certificates managed in ACM have no additional charge.

4) VPC + private subnets

What it does: isolates your app network from the public internet.

Why it is there: keeps the EC2 instances private while the ALB stays public.

What it prevents: direct public access to your app servers.

Cost: no additional charge for using a VPC.

5) Security groups / NACL layer

What it does: acts like a firewall for what can talk to what.

Why it is there: only the ALB should reach the app, and only the app should reach allowed outbound services.

What it prevents: exposed ports, random inbound access, and accidental wide-open networking.

Cost: no additional charge for security groups; they are part of the VPC control layer in this design.

6) Auto Scaling Group (ASG)

What it does: launches more EC2 instances when demand rises and replaces unhealthy ones.

Why it is there: it is the core of the project.

What it prevents: traffic spikes from taking down one server, and failed instances from staying dead.

Cost: no additional fee for Auto Scaling itself; you pay only for the instances it launches and the monitoring it uses.

7) EC2 instances

What it does: runs the actual application.

Why it is there: the load balancer needs compute targets.

What it prevents: a static-only architecture and a single-server setup.

Cost: t3.micro is \$0.0104/hr in the pricing example shown by AWS; one instance left on all month is about \$7.49 before storage and traffic.

8) IAM role for the EC2 instances

What it does: gives the instance permission to talk to AWS services without hard-coded keys.

Why it is there: your app can write logs, read S3, or be managed securely.

What it prevents: leaked access keys inside the server.

Cost: no additional charge.

9) Systems Manager Session Manager (SSM)

What it does: lets you connect to the instance through AWS without opening SSH to the world.

Why it is there: this is one of the strongest portfolio-security upgrades you can show.

What it prevents: open port 22, bastion-host cost, and long-lived SSH key handling.

Cost: no additional charge for Systems Manager; you only pay for the AWS resources it uses.

10) CloudWatch metrics + alarms

What it does: monitors CPU/health and triggers scale-out or scale-in.

Why it is there: makes the autoscaling behavior visible and believable.

What it prevents: scaling by guesswork and slow reaction to load.

Cost: a standard metric alarm is about \$0.10 per alarm-metric per month. For a small project with just a couple of alarms, this is usually only pennies.

11) S3 for static assets, logs, or backup files

What it does: stores files outside the instance.

Why it is there: useful for logs, frontend assets, or simple backups.

What it prevents: losing important files when an instance is replaced.

Cost: S3 Standard is \$0.023 per GB-month. For a tiny portfolio bucket, this is usually very cheap.

12) EBS root volume / backups

What it does: gives each instance its boot disk and stores local server data.

Why it is there: the instance needs a filesystem to run from.

What it prevents: losing the OS disk and app files immediately when the instance is terminated.

Cost: gp3 storage is \$0.08 per GB-month, and standard EBS snapshots are \$0.05 per GB-month. A small 8–10 GB demo volume is usually only a few cents to well under a dollar per month.

13) CloudFront (optional)

What it does: adds a CDN layer in front of your app or static content.

Why it is there: it makes the project look even more polished and can reduce origin load.

What it prevents: slower global access and extra load hitting the origin directly.

Cost: CloudFront has a \$0/month Free plan; paid plans add more features, so this is a good optional upgrade only if you want the extra polish.

For a 4-hour demo, the whole stack is usually very cheap because the free/no-fee pieces dominate and the paid pieces only run briefly. The main charges are EC2 and the ALB; everything else in this design is free or very small at portfolio scale.